

手摸手之代码格式化很重要

为什么要代码格式化？代码格式化的意义是让代码更加**易读易懂易修改**。

ShardingSphere 作为 Apache 顶级开源项目，截止当前已有 380+ 贡献者。因为大部分开发人员的代码风格不一致，在 Github 多人协作的模式下，不易保障项目整体代码格式。

基于以上诉求，ShardingSphere 采用了 [Spotless](#) 充当代码格式统一的角色。

什么是 Spotless

Spotless 是支持多种语言的代码格式化工具，支持 Maven 和 Gradle 以 Plugin 的形式构建。

Spotless 对开发者来说, 有 2 种使用方式: 检查代码是否存在格式问题, 以及格式化代码。

如何使用

下述代码是 Spotless 官方示例。

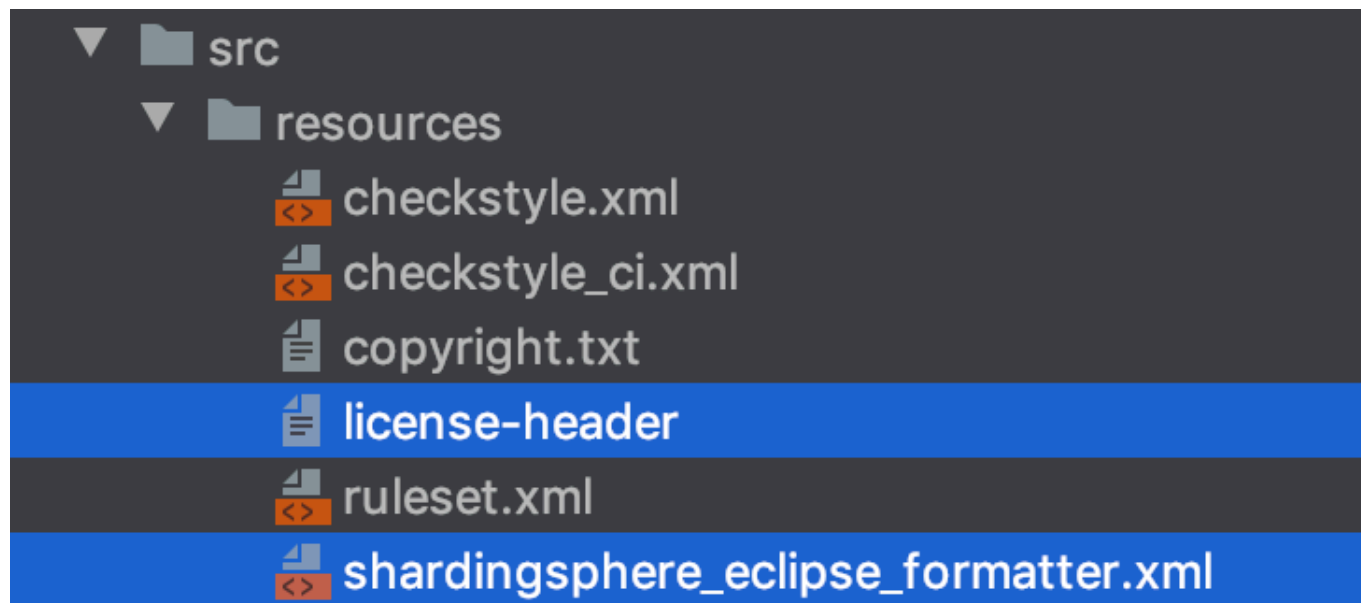
```
user@machine repo % mvn spotless:check
[ERROR] > The following files had format violations:
[ERROR]   src\main\java\com\diffplug\gradle\spotless\FormatExtension.java
[ERROR]     -\t\t...if.(targets.length==.0).{
[ERROR]       +\t\tif.(targets.length==.0).{
[ERROR]   Run 'mvn spotless:apply' to fix these violations.
user@machine repo % mvn spotless:apply
[INFO] BUILD SUCCESS
user@machine repo % mvn spotless:check
[INFO] BUILD SUCCESS
```

通过 `mvn spotless:check` 检查项目代码时发现错误，接着使用 `mvn spotless:apply` 进行代码格式化；再次检查时，格式化错误消失。

1. 项目实战

ShardingSphere 使用 Spotless 实现了添加 **Java 文件 licenseHeader** 和 **Java 代码格式化**。

Spotless 有多种 Java 代码格式化方式，例如：googleJavaFormat、eclipse、prettier 等。基于定制化的考虑，最终采用 eclipse 进行 Java 代码格式化。



1) 根据项目需求添加 licenseHeader

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version 2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *     http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
```

注意，licenseHeader **最后要留有一行空格**。不然 licenseHeader 和 package 之间将没有空隙。

2) 添加 shardingsphere_eclipse_formatter.xml

```

<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!--
/**
 * Licensed to the Apache Software Foundation (ASF) under one
 * or more contributor license agreements. See the NOTICE file
 * distributed with this work for additional information
 * regarding copyright ownership. The ASF licenses this file
 * to you under the Apache License, Version 2.0 (the
 * "License"); you may not use this file except in compliance
 * with the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
-->
<profiles version="13">
  <profile kind="CodeFormatterProfile" name="'ShardingSphere Apache Current'"
version="13">
    <setting id="org.eclipse.jdt.core.compiler.source" value="1.8"/>
    <setting id="org.eclipse.jdt.core.compiler.compliance" value="1.8"/>
    <setting id="org.eclipse.jdt.core.compiler.codegen.targetPlatform" value="1.8"/>
    <setting id="org.eclipse.jdt.core.formatter.indent_empty_lines" value="true"/>
    <setting id="org.eclipse.jdt.core.formatter.tabulation.size" value="4"/>
    <setting id="org.eclipse.jdt.core.formatter.lineSplit" value="200"/>
    <setting id="org.eclipse.jdt.core.formatter.comment.line_length" value="200"/>
    <setting id="org.eclipse.jdt.core.formatter.tabulation.char" value="space"/>
    <setting id="org.eclipse.jdt.core.formatter.indentation.size" value="1"/>
    <setting id="org.eclipse.jdt.core.formatter.comment.format_javadoc_comments"
value="false"/>
    <setting id="org.eclipse.jdt.core.formatter.join_wrapped_lines" value="false"/>
    <setting
id="org.eclipse.jdt.core.formatter.insert_space_before_colon_in_conditional"
value="insert"/>
    <setting id="org.eclipse.jdt.core.formatter.insert_space_before_colon_in_default"
value="do not insert"/>
    <setting id="org.eclipse.jdt.core.formatter.alignment_for_enum_constants"

```

```
value="16"/>
    <setting
id="org.eclipse.jdt.core.formatter.insert_space_before_colon_in_labeled_statement"
value="do not insert"/>
    <setting id="org.eclipse.jdt.core.formatter.insert_space_before_colon_in_case"
value="do not insert"/>
    <setting id="org.eclipse.jdt.core.formatter.alignment_for_conditional_expression"
value="80"/>
    <setting id="org.eclipse.jdt.core.formatter.alignment_for_assignment"
value="16"/>
    <setting id="org.eclipse.jdt.core.formatter.blank_lines_after_package"
value="1"/>
    <setting
id="org.eclipse.jdt.core.formatter.continuation_indentation_for_array_initializer"
value="2"/>
    <setting id="org.eclipse.jdt.core.formatter.alignment_for_resources_in_try"
value="160"/>
    <setting
id="org.eclipse.jdt.core.formatter.alignment_for_throws_clause_in_method_declaration"
value="10"/>
    <setting
id="org.eclipse.jdt.core.formatter.alignment_for_parameters_in_method_declaration"
value="106"/>
    <setting
id="org.eclipse.jdt.core.formatter.alignment_for_parameters_in_constructor_declaration"
value="106"/>
    <setting
id="org.eclipse.jdt.core.formatter.alignment_for_throws_clause_in_constructor_declaration"
value="106"/>
    <setting
id="org.eclipse.jdt.core.formatter.alignment_for_arguments_in_explicit_constructor_call.c
ount_dependent" value="16|5|80"/>
</profile>
</profiles>
```

ShardingSphere 最新规则查看 [shardingsphere_eclipse_formatter.xml](#), 另提供一份 [eclipse-java-google-style.xml](#) 供参考

shardingsphere_eclipse_formatter.xml 的内容是根据 ShardingSphere 代码规范定制开发的，可灵活变动。

3) 添加 Maven plugin

```
<plugin>
  <groupId>com.diffplug.spotless</groupId>
  <artifactId>spotless-maven-plugin</artifactId>
  <version>2.22.1</version>
  <configuration>
    <java>
      <eclipse>

<file>${maven.multiModuleProjectDirectory}/src/resources/shardingsphere_eclipse_formatter.xml</file>

      </eclipse>
      <licenseHeader>
        <file>${maven.multiModuleProjectDirectory}/src/resources/license-header</file>
      </licenseHeader>
    </java>
  </configuration>
</plugin>
```

Spotless 支持格式化指定目录，以及排除指定目录的功能，详情参考 [plugin-maven#java](#)。如无指定，执行 check 或 apply 时，默认项目全量代码。

4) 执行代码格式化

执行完上述三个步骤，就可以在项目中执行命令，检查 Java 代码是否符合规范，以及代码格式化功能。

```
user@machine repo % mvn spotless:apply
[INFO] BUILD SUCCESS
user@machine repo % mvn spotless:check
[INFO] BUILD SUCCESS
```

2. 绑定 Maven 生命周期

在 ShardingSphere 实际应用中，选择将 Spotless apply 绑定到 compile 阶段，这样本地执行 mvn install 时就能自动格式化。

```
<plugin>
  <groupId>com.diffplug.spotless</groupId>
  <artifactId>spotless-maven-plugin</artifactId>
  <version>2.22.1</version>
  <configuration>
    <java>
      <eclipse>

<file>${maven.multiModuleProjectDirectory}/src/resources/shardingsphere_eclipse_formatter
.xml</file>

      </eclipse>
      <licenseHeader>
        <file>${maven.multiModuleProjectDirectory}/src/resources/license-
header</file>
      </licenseHeader>
    </java>
  </configuration>
  <executions>
    <execution>
      <goals>
        <goal>apply</goal>
      </goals>
      <phase>compile</phase>
    </execution>
  </executions>
</plugin>
```

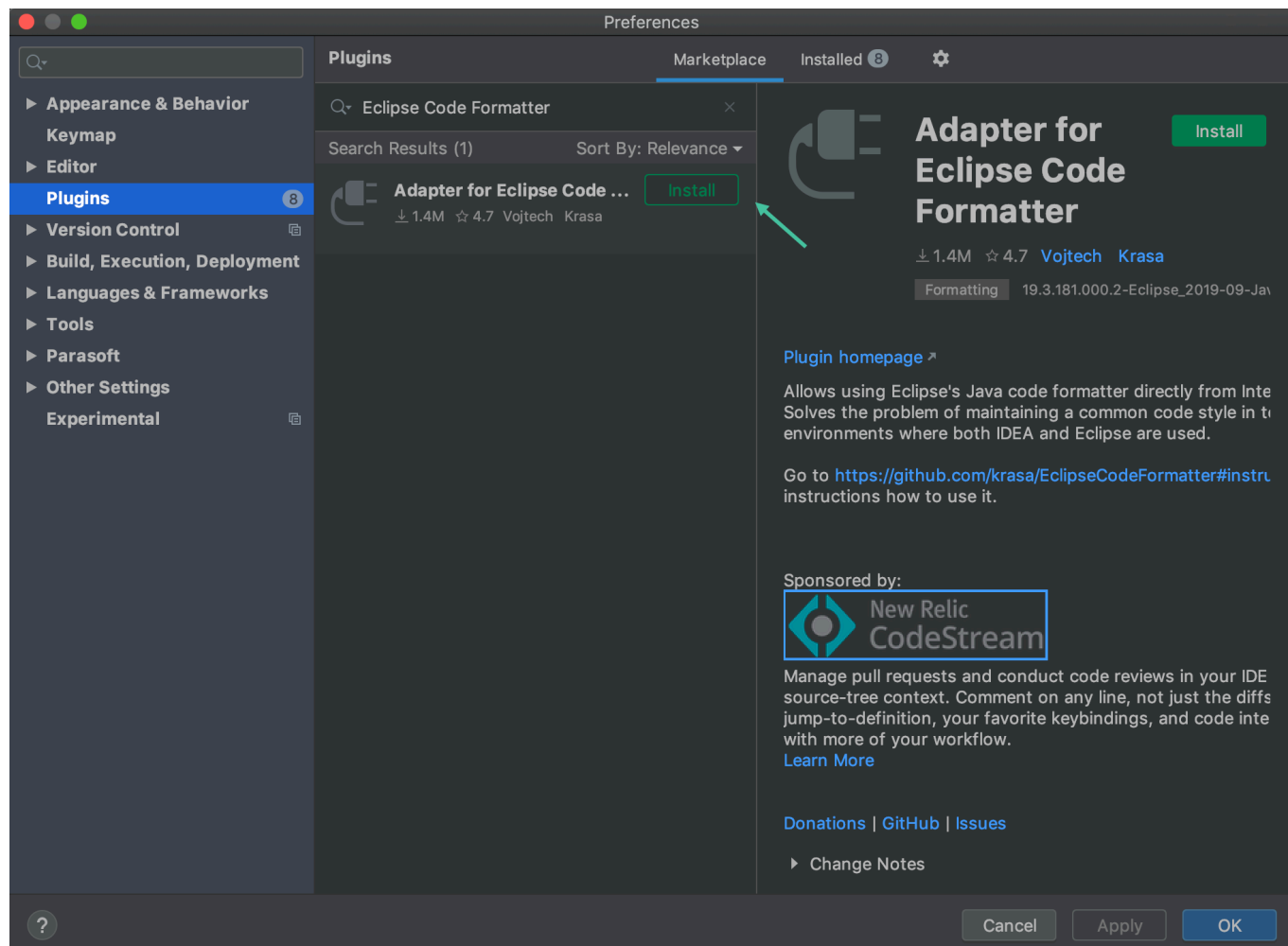
3. IDEA 格式化

开发者如果在写代码过程中，想检查单个文件是否符合规范，执行 `mvn spotless:check` 或 `mvn spotless:apply` 显得有些笨重，因为格式化范围默认是整个项目。

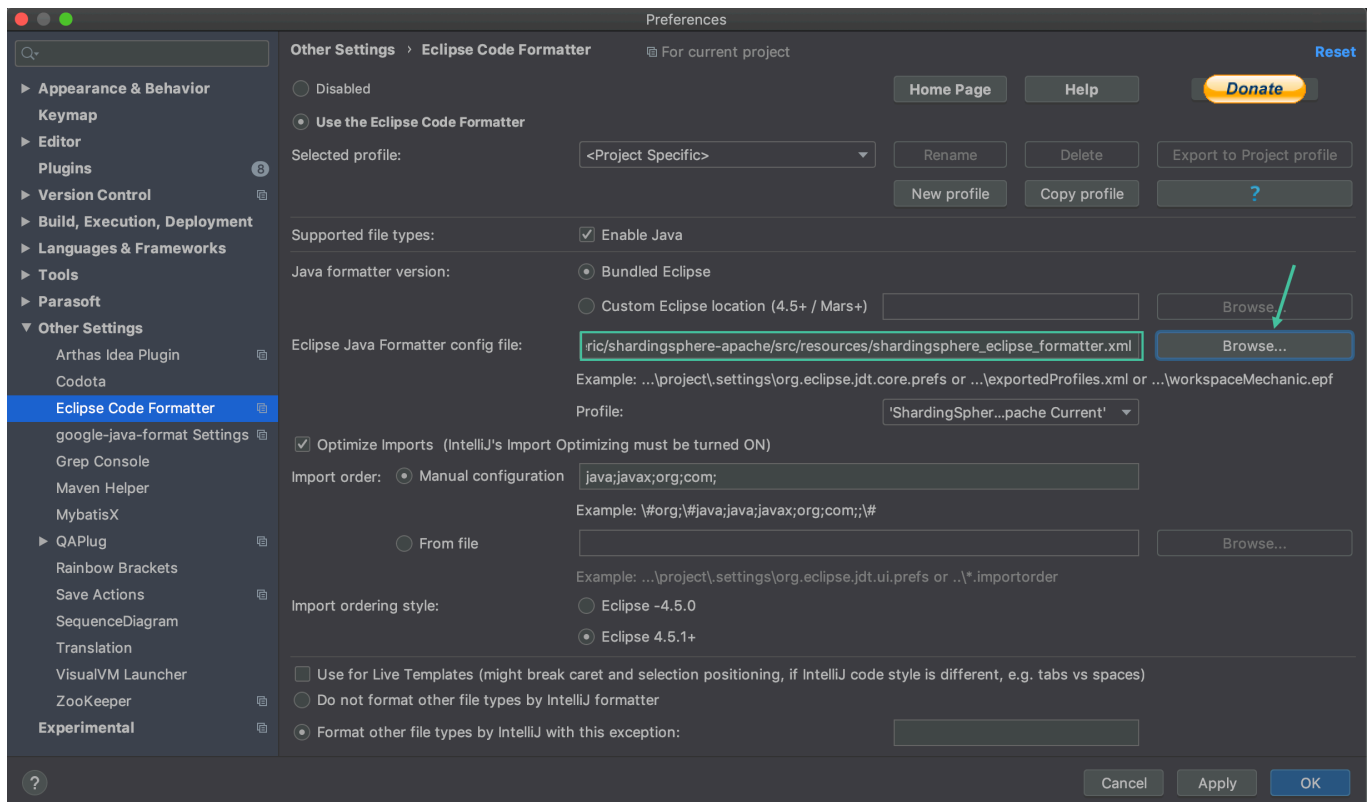
我们可以使用 `shardingsphere\_eclipse\_formatter.xml` 替换 IntelliJ IDEA 原有格式化功能，这样在写代码过程中可以随时格式化，极大提升了开发效率。

IDEA 版本 2019.3.4。

1) 安装插件 Eclipse Code Formatter



2) 选择 shardingsphere_eclipse_formatter.xml 为默认格式化模板



使用 IDEA 代码格式化快捷键，就可以完成 Spotless 代码格式化。

常见问题

1. Spotless 与 Checkstyle 冲突

Checkstyle 是一种用于检查 Java 源代码是否符合代码标准或一组验证规则（最佳实践）的工具。

极端场景下，Spotless 格式化代码后，通过 Checkstyle 检查代码会不通过。

根本原因在于两者设定的检查配置和格式化配置冲突。举个例子，Spotless 格式化后换行缩进了 16 个空格，而 Checkstyle 的换行检查是 12 个空格。

```
private static Collection<PreciseHintShadowValue<Comparable<?>>>>
createNoteShadowValues(final ShadowDetermineCondition shadowDetermineCondition) {
    // Checkstyle 可以通过的格式
    return shadowDetermineCondition.getSqlComments().stream().
    <PreciseHintShadowValue<Comparable<?>>>>map(
        each -> new PreciseHintShadowValue<>(tableName, shadowOperationType,
```



```
each)).collect(Collectors.toList());  
    // Spotless 格式化后  
    return shadowDetermineCondition.getSqlComments().stream().  
    <PreciseHintShadowValue<Comparable<?>>>map(  
        each -> new PreciseHintShadowValue<>(tableName, shadowOperationType,  
        each)).collect(Collectors.toList());  
    }
```

这种情况，需要开发者衡量如何取舍。解决方案有两种：修改 Spotless 的格式化规则，或修改 Checkstyle 的检查规则。

2. CRLF 与 LF 格式化冲突

参考 <<https://github.com/diffplug/spotless/issues/1171>>

文末总结

文章介绍了 Apache ShardingSphere 使用 Spotless 完成历史代码格式化，以及后续代码格式的统一，对项目代码的整洁起到了很大帮助。

当然，Spotless 的功能不止 Java 代码的格式化，包括不限于 Pom 和 Markdown 等文件类型的格式化，后续这些都会在 ShardingSphere 得以应用。

更新: 2023-07-17 17:47:46

原文: <<https://www.yuque.com/magestack/12306/pf7g1vgroinnz98o>>